

CV-12: Camera Image Rectification and Preprocessing

1. Introduction

The purpose of this task was to configure and validate camera image rectification for the Team 7 computer vision pipeline. The main objective was to ensure that the camera image and corresponding camera_info calibration data could be processed using the standard ROS 2 image rectification pipeline before being passed to the object detection system.

Another important objective was to keep the camera topics configurable. This allows the same computer vision pipeline to work with the current Gazebo simulated camera and later be adapted to a physical camera without rewriting the detector.

2. Initial Camera Verification

The Team 7 Gazebo simulation was started and the preflight check completed successfully with 20 checks passed and 0 failed. Both the top and side camera streams were available and producing valid image content.

The overhead camera topics were checked using:

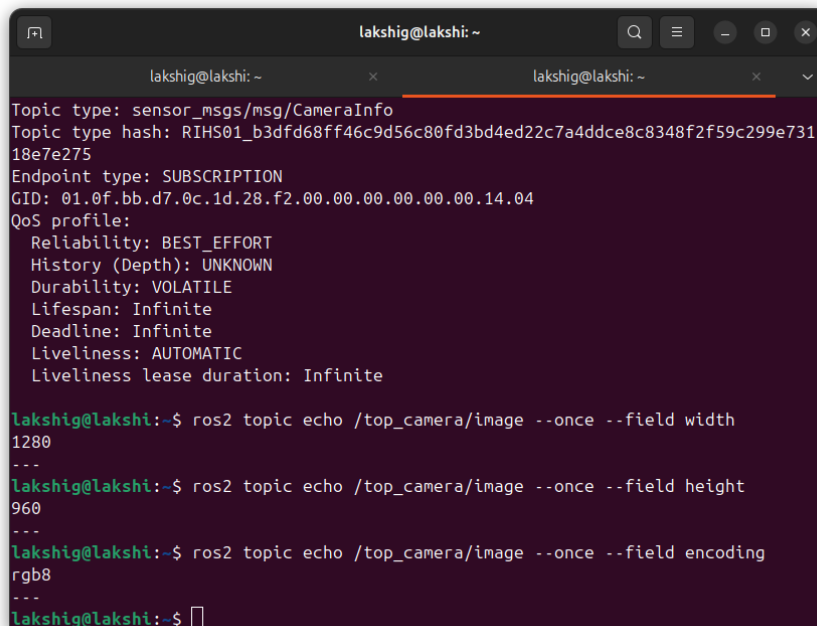
```
ros2 topic list | grep top_camera
```

The following topics were available:

```
/top_camera/camera_info
```

```
/top_camera/image
```

Further inspection confirmed that /top_camera/image had one publisher from top_camera_bridge.



```
lakshig@lakshi: ~  
Topic type: sensor_msgs/msg/CameraInfo  
Topic type hash: RIHS01_b3dfd68ff46c9d56c80fd3bd4ed22c7a4ddce8c8348f2f59c299e73118e7e275  
Endpoint type: SUBSCRIPTION  
GID: 01.0f.bb.d7.0c.1d.28.f2.00.00.00.00.00.14.04  
QoS profile:  
  Reliability: BEST_EFFORT  
  History (Depth): UNKNOWN  
  Durability: VOLATILE  
  Lifespan: Infinite  
  Deadline: Infinite  
  Liveliness: AUTOMATIC  
  Liveliness lease duration: Infinite  
  
lakshig@lakshi:~$ ros2 topic echo /top_camera/image --once --field width  
1280  
---  
lakshig@lakshi:~$ ros2 topic echo /top_camera/image --once --field height  
960  
---  
lakshig@lakshi:~$ ros2 topic echo /top_camera/image --once --field encoding  
rgb8  
---  
lakshig@lakshi:~$
```

The raw overhead camera image had the following properties:

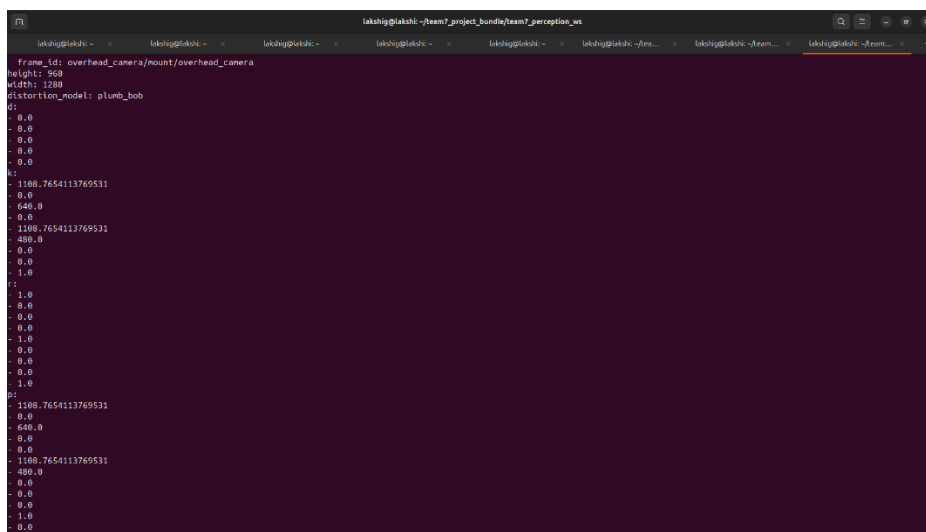
Width: 1280
Height: 960
Encoding: rgb8
Frame ID: overhead_camera/mount/overhead_camera

The camera image was also visually inspected to confirm that the Gazebo scene was visible and the image was not black.

3. Camera Calibration Verification

The calibration information was inspected using:

```
ros2 topic echo /top_camera/camera_info -once
```



```
lakshig@lakshi:~/team7_project_bondia/team7_perception_ws
Frame ID: overhead_camera/mount/overhead_camera
height: 960
width: 1280
distortion_model: plumb_bob
D:
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
K:
- 1108.7654113769531
- 0.0
- 640.0
- 0.0
- 1108.7654113769531
- 480.0
- 0.0
- 0.0
- 1.0
r:
- 1.0
- 0.0
- 0.0
- 0.0
- 1.0
- 0.0
- 0.0
- 0.0
- 1.0
p:
- 1108.7654113769531
- 0.0
- 640.0
- 0.0
- 1108.7654113769531
- 480.0
- 0.0
- 0.0
- 0.0
- 0.0
- 1.0
- 0.0
```

The CameraInfo message reported:

Resolution: 1280 × 960
Distortion model: plumb_bob
D: [0, 0, 0, 0, 0]
fx: 1108.7654
fy: 1108.7654
cx: 640
cy: 480

The intrinsic camera matrix K, rectification matrix R, and projection matrix P were also populated correctly.

All five distortion coefficients were zero. Therefore, the simulated Gazebo camera does not introduce lens distortion. As a result, the raw and rectified images were expected to appear almost identical.

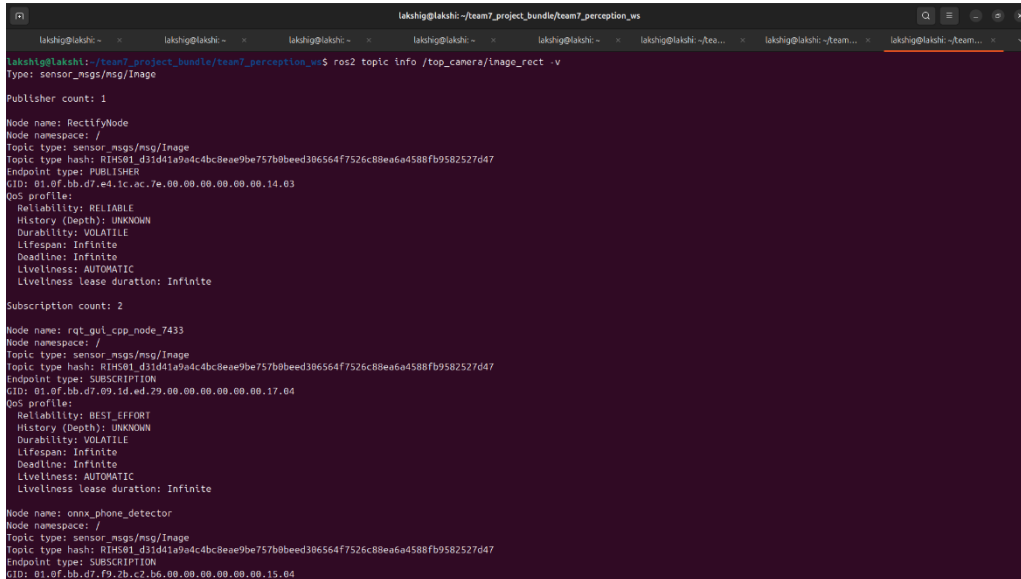
The `roi.do_rectify` field was shown as `false`. This does not mean that image rectification was disabled. This field belongs to the CameraInfo Region of Interest configuration and indicates

```
ros2 pkg executables image_proc | grep rect
```

5. Rectified Image Validation

The rectified image topic was inspected using:

```
ros2 topic info /top_camera/image_rect -v
```



```
lakshig@lakshi:~/team7_project_bundle/team7_perception_ws$ ros2 topic info /top_camera/image_rect -v
Type: sensor_msgs/msg/Image
Publisher count: 1
Node name: RectifyNode
Node namespace: /
Topic type: sensor_msgs/msg/Image
Topic type hash: RIH501_d31d1a9a4c4bc8ee9be757b0beed306564f7526c88ea6a4588fb9582527d47
Endpoint type: PUBLISHER
CID: 01.0f.bb.d7.e4.1c.ac.7e.00.00.00.00.00.14.03
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
Subscription count: 2
Node name: rqt_gui_cpp_node_7433
Node namespace: /
Topic type: sensor_msgs/msg/Image
Topic type hash: RIH501_d31d1a9a4c4bc8ee9be757b0beed306564f7526c88ea6a4588fb9582527d47
Endpoint type: SUBSCRIPTION
CID: 01.0f.bb.d7.09.1d.ed.29.00.00.00.00.00.17.04
QoS profile:
  Reliability: BEST_EFFORT
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
Node name: onnx_phone_detector
Node namespace: /
Topic type: sensor_msgs/msg/Image
Topic type hash: RIH501_d31d1a9a4c4bc8ee9be757b0beed306564f7526c88ea6a4588fb9582527d47
Endpoint type: SUBSCRIPTION
CID: 01.0f.bb.d7.f9.2b.c2.b6.00.00.00.00.00.15.04
```

The output confirmed:

Type: sensor_msgs/msg/Image

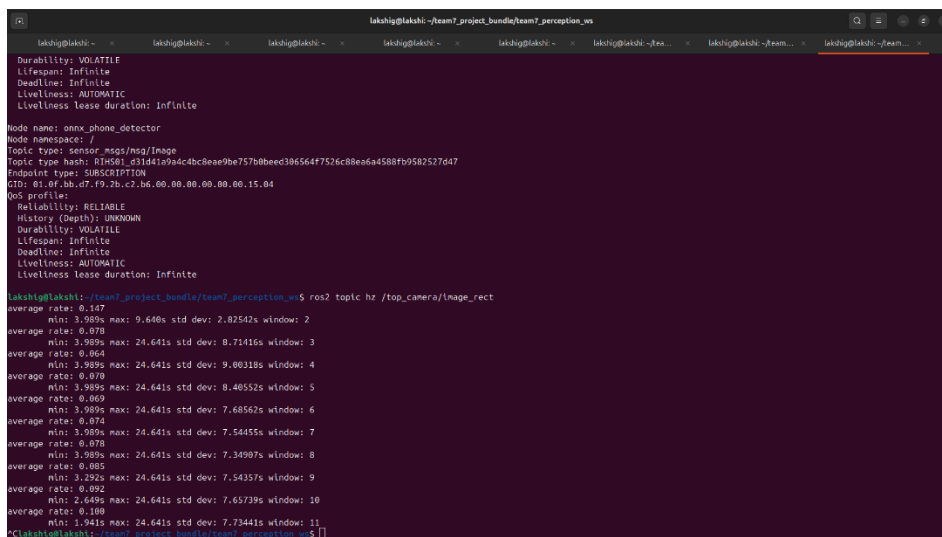
Publisher count: 1

Node name: RectifyNode

Therefore, the standard ROS RectifyNode was successfully publishing the rectified camera stream.

The publishing rate was also checked using:

```
ros2 topic hz /top_camera/image_rect
```



```
lakshig@lakshi:~/team7_project_bundle/team7_perception_ws$ ros2 topic hz /top_camera/image_rect
average rate: 0.147
  min: 3.989s max: 9.640s std dev: 2.82542s window: 2
average rate: 0.078
  min: 3.989s max: 24.641s std dev: 8.73416s window: 3
average rate: 0.064
  min: 3.989s max: 24.641s std dev: 9.00318s window: 4
average rate: 0.070
  min: 3.989s max: 24.641s std dev: 8.40552s window: 5
average rate: 0.060
  min: 3.989s max: 24.641s std dev: 7.68562s window: 6
average rate: 0.074
  min: 3.989s max: 24.641s std dev: 7.54455s window: 7
average rate: 0.078
  min: 3.989s max: 24.641s std dev: 7.34907s window: 8
average rate: 0.065
  min: 3.292s max: 24.641s std dev: 7.54357s window: 9
average rate: 0.092
  min: 2.649s max: 24.641s std dev: 7.65739s window: 10
average rate: 0.100
  min: 1.941s max: 24.641s std dev: 7.73441s window: 11
^Clakshig@lakshi:~/team7_project_bundle/team7_perception_ws$
```

Live image frames were received, although the observed rate varied under the simulation workload.

The rectified image header was checked and retained the correct camera frame:

```
frame_id: overhead_camera/mount/overhead_camera
```

Both `/top_camera/image` and `/top_camera/image_rect` were visually inspected. No unexpected distortion or image corruption was observed. The images appeared almost identical, which was expected because the Gazebo camera's distortion coefficients were all zero.

6. Integration with the Computer Vision Pipeline

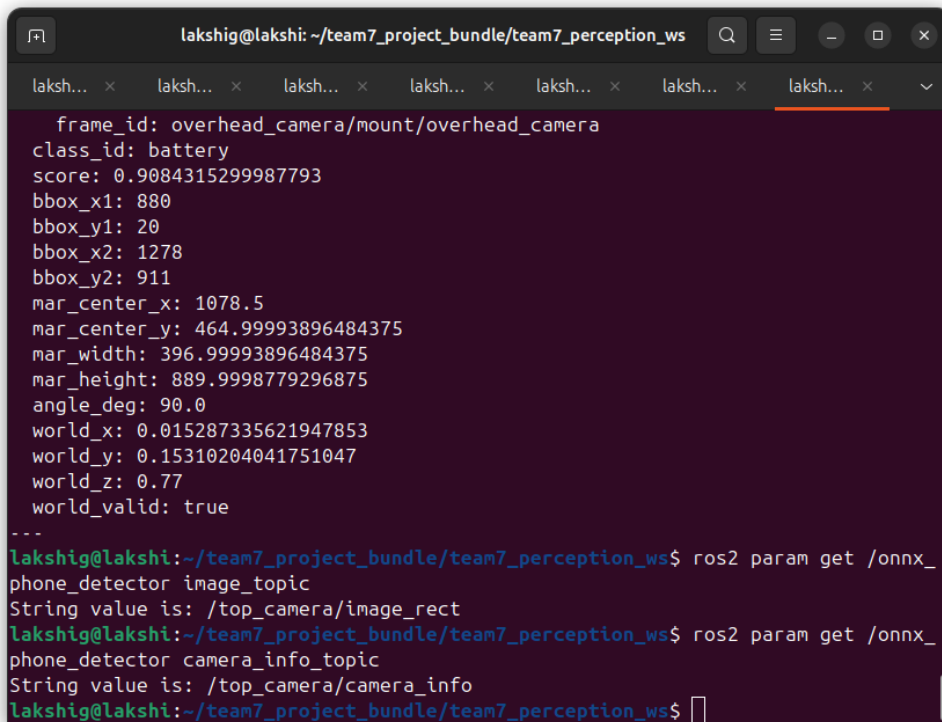
After validating the rectified image, the existing ONNX phone detector was configured to consume the rectified image instead of the raw image.

The detector started with:

```
python -m team7_perception.onnx_phone_detector_node --ros-args \
  -p image_topic:=/top_camera/image_rect \
  -p camera_info_topic:=/top_camera/camera_info
```

The detector successfully loaded the YOLO11 segmentation ONNX model and reported:

```
onnx_phone_detector up:
/top_camera/image_rect -> /phone_detections
```

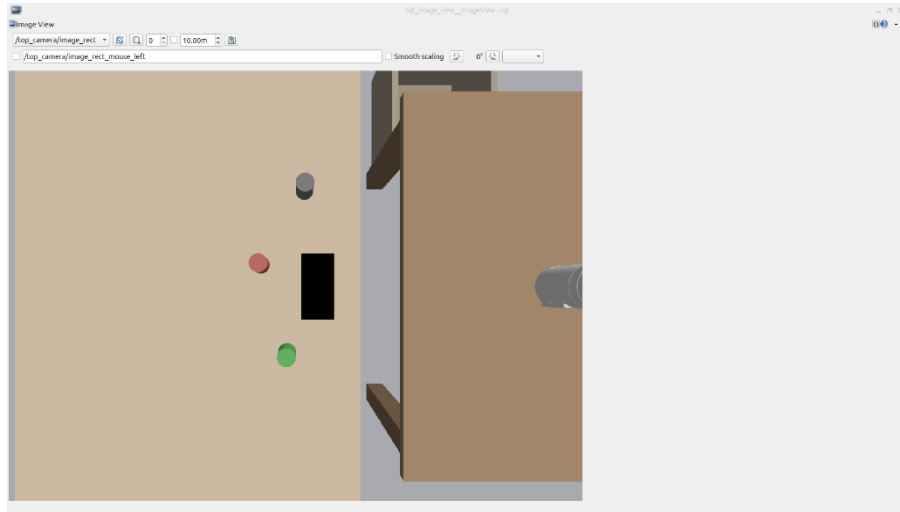


```
lakshig@lakshi: ~/team7_project_bundle/team7_perception_ws
frame_id: overhead_camera/mount/overhead_camera
class_id: battery
score: 0.9084315299987793
bbox_x1: 880
bbox_y1: 20
bbox_x2: 1278
bbox_y2: 911
mar_center_x: 1078.5
mar_center_y: 464.99993896484375
mar_width: 396.99993896484375
mar_height: 889.9998779296875
angle_deg: 90.0
world_x: 0.015287335621947853
world_y: 0.15310204041751047
world_z: 0.77
world_valid: true
---
lakshig@lakshi:~/team7_project_bundle/team7_perception_ws$ ros2 param get /onnx_
phone_detector image_topic
String value is: /top_camera/image_rect
lakshig@lakshi:~/team7_project_bundle/team7_perception_ws$ ros2 param get /onnx_
phone_detector camera_info_topic
String value is: /top_camera/camera_info
lakshig@lakshi:~/team7_project_bundle/team7_perception_ws$
```

The detector parameters were independently verified using:

```
ros2 param get /onnx_phone_detector image_topic  
ros2 param get /onnx_phone_detector camera_info_topic
```

The results were:



```
image_topic:  
/top_camera/image_rect
```

```
lakshig@lakshi:~/team7_project_bundle/team7_perception_ws$ ros2 topic info /top_camera/image_rect -v  
Type: sensor_msgs/msg/Image  
Publisher count: 1  
Node name: RectifyNode  
Node namespace: /  
Topic type: sensor_msgs/msg/Image  
Topic type hash: RIHS01_d31d41a9e4c4bc8eae9be757b0beed306564f7526c88e6a4588fb9582527d47  
Endpoint type: PUBLISHER  
GID: 01.0f.bb.d7.e4.1c.ac.7e.00.00.00.00.00.14.03  
QoS profile:  
Reliability: RELIABLE  
History (Depth): UNKNOWN  
Durability: VOLATILE  
Lifespan: Infinite  
Deadline: Infinite  
Liveliness: AUTOMATIC  
Liveliness lease duration: Infinite  
Subscription count: 2  
Node name: rqt_gui_cpp_node_7433  
Node namespace: /  
Topic type: sensor_msgs/msg/Image  
Topic type hash: RIHS01_d31d41a9e4c4bc8eae9be757b0beed306564f7526c88e6a4588fb9582527d47  
Endpoint type: SUBSCRIPTION  
GID: 01.0f.bb.d7.09.1d.ed.29.00.00.00.00.00.17.04  
QoS profile:  
Reliability: BEST_EFFORT  
History (Depth): UNKNOWN  
Durability: VOLATILE  
Lifespan: Infinite  
Deadline: Infinite  
Liveliness: AUTOMATIC  
Liveliness lease duration: Infinite  
Node name: onnx_phone_detector  
Node namespace: /  
Topic type: sensor_msgs/msg/Image  
Topic type hash: RIHS01_d31d41a9e4c4bc8eae9be757b0beed306564f7526c88e6a4588fb9582527d47  
Endpoint type: SUBSCRIPTION  
GID: 01.0f.bb.d7.f9.2b.c2.b6.00.00.00.00.00.15.04
```

```
camera_info_topic:  
/top_camera/camera_info
```

This confirmed that the detector was consuming the rectified image while continuing to use the corresponding calibration information.

```
lakshig@lakshi: ~ - lakshig@lakshi - lakshig@lakshi - lakshig@lakshi - lakshig@lakshi - lakshig@lakshi - /team7_perception_ws
Topic type hash: RIHS01_d31d41a9e4c4bc8eae9be757b0beed386564f7526c88eaa4588fb9582527d47
Endpoint type: PUBLISHER
CID: 01.0f.bb.d7.e4.1c.ac.7e.00.00.00.00.00.14.03
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
Subscription count: 2
Node name: rqt_gui_cpp_node_7433
Node namespace: /
Topic type: sensor_msgs/Image
Topic type hash: RIHS01_d31d41a9e4c4bc8eae9be757b0beed386564f7526c88eaa4588fb9582527d47
Endpoint type: SUBSCRIPTION
CID: 01.0f.bb.d7.09.1d.ed.29.00.00.00.00.00.17.04
QoS profile:
  Reliability: BEST_EFFORT
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
Node name: onnx_phone_detector
Node namespace: /
Topic type: sensor_msgs/Image
Topic type hash: RIHS01_d31d41a9e4c4bc8eae9be757b0beed386564f7526c88eaa4588fb9582527d47
Endpoint type: SUBSCRIPTION
CID: 01.0f.bb.d7.f9.2b.c2.b6.00.00.00.00.00.15.04
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
```

7. Detection Validation

The detector output was checked using:

```
ros2 topic echo /phone_detections
```

```
lakshig@lakshi: ~ - lakshig@lakshi - lakshig@lakshi - lakshig@lakshi - lakshig@lakshi - lakshig@lakshi - /team7_perception_ws
frame_id: overhead_camera/mount/overhead_camera
detections:
- header:
  stamp:
    sec: 3896
    nanosec: 508008060
  frame_id: overhead_camera/mount/overhead_camera
  class_id: battery
  score: 0.9084315299987793
  bbox_x1: 800
  bbox_y1: 20
  bbox_x2: 1278
  bbox_y2: 911
  mar_center_x: 1078.5
  mar_center_y: 464.99993896484375
  mar_width: 396.99993896484375
  mar_height: 889.9998779296875
  angle_deg: 90.0
  world_x: 0.01528733621947053
  world_y: 0.1531620484751047
  world_z: 0.77
  world_valid: true
- header:
  stamp:
    sec: 3897
    nanosec: 0
  frame_id: overhead_camera/mount/overhead_camera
  detections:
  - header:
    stamp:
      sec: 3897
      nanosec: 0
    frame_id: overhead_camera/mount/overhead_camera
    class_id: battery
    score: 0.9084315299987793
    bbox_x1: 800
    bbox_y1: 20
    bbox_x2: 1278
    bbox_y2: 911
    mar_center_x: 1078.5
    mar_center_y: 464.99993896484375
    mar_width: 396.99993896484375
    mar_height: 889.9998779296875
    angle_deg: 90.0
    world_x: 0.01528733621947053
    world_y: 0.1531620484751047
    world_z: 0.77
    world_valid: true
```

Live detection messages were successfully received. For example, the detector identified a battery with a confidence score of approximately 0.91 and produced bounding-box coordinates in the source-image coordinate system.

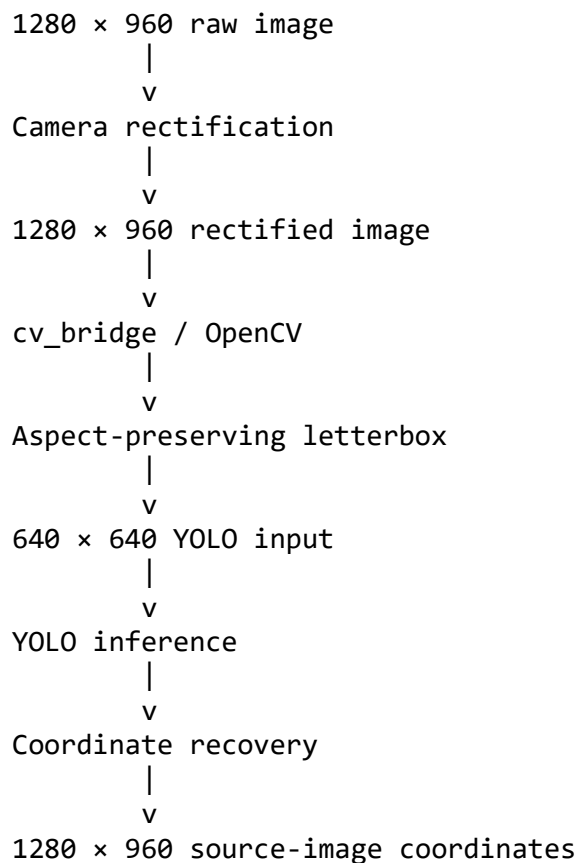
This confirmed that introducing the rectification stage did not break the existing object detection pipeline.

A geometry self-check warning was also observed for the calculated world coordinates. This issue relates to the separate pixel-to-world localisation work and was not modified as part of CV-12. The successful image detection and bounding-box output were used for validating the rectification task.

8. Image Preprocessing

The existing perception pipeline already performs the required model preprocessing. The rectified ROS image is converted to an OpenCV BGR image using `cv_bridge`. The image is then letterboxed while maintaining its aspect ratio and converted to the 640×640 input required by the YOLO ONNX model.

The processing sequence is therefore:



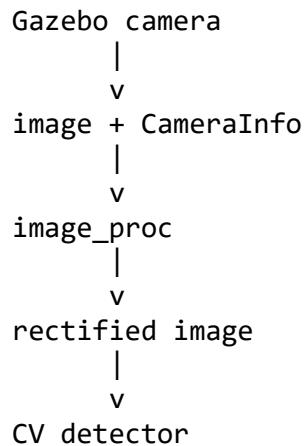
Camera rectification and model letterboxing are separate operations. Rectification corrects camera geometry using calibration information, while letterboxing prepares the image for neural-network inference.

No changes to the existing inference preprocessing were required for this task.

9. Future Physical-Camera Compatibility

A major benefit of the implemented architecture is that the detector does not depend directly on the topic of Gazebo raw image.

The current pipeline is:



A future physical camera can provide its own raw image and calibrated CameraInfo. These topics can then be remapped into the same rectification pipeline, while the detector continues consuming a rectified image topic.

This reduces the amount of code that needs to change when moving from simulation to physical hardware.

10. GitHub Documentation and Version Control

As part of this task, the camera image rectification work was also documented and added to GitHub for version control. A new repository named redback-e-waste-battery-extraction-computer-vision was created to maintain the computer vision-related development work.

A reusable ROS 2 launch file, camera_rectification.launch.py, was created inside the team7_perception package. The launch file starts the image_proc rectification node and provides configurable topic arguments for the raw camera image, camera calibration information, and rectified image output. This avoids hard-coding the camera topics and makes the implementation easier to adapt for Gazebo cameras and future physical-camera integration.

The setup.py file was updated so that the new launch file is installed correctly when the ROS 2 workspace is built. The package.xml file was also updated to include the required runtime dependencies for launch, launch_ros, and image_proc.

The launch file was then rebuilt and tested in the Team 7 simulation environment. The camera_rectifier node successfully subscribed to /top_camera/image and /top_camera/camera_info and published /top_camera/image_rect. Topic inspection confirmed one publisher for the rectified image, and an image message was successfully received with the expected frame ID overhead_camera/mount/overhead_camera. The observed image rate was low and irregular because the raw Gazebo camera stream was also running at a low and irregular rate under the simulation workload.

Only the files related to my camera rectification contribution were committed so that unrelated work completed by other team members was not included. The committed files were .gitignore, camera_rectification.launch.py, package.xml, and setup.py. The changes were committed with the message “Add configurable camera image rectification” and successfully pushed to the main branch of the GitHub repository. This provides version-controlled evidence of the implementation and makes the rectification configuration available for future development and integration.

[redback-e-waste-battery-extraction-computer-vision/src/team7_perception at main · lakshig/redback-e-waste-battery-extraction-computer-vision](#)

11. Conclusion

The camera image rectification pipeline was successfully configured and validated for the Team 7 Gazebo overhead camera. The raw 1280×960 image and corresponding CameraInfo were supplied to the standard ROS 2 image_proc rectification node, which successfully produced /top_camera/image_rect.

Visual inspection confirmed that rectification did not introduce unexpected image distortion. The raw and rectified images appeared almost identical because the current Gazebo camera has zero distortion coefficients.

The existing YOLO detector was successfully configured to consume /top_camera/image_rect, and live object detections continued to be published through /phone_detections. The implementation therefore provides a working rectification stage while maintaining configurable ROS topics for future physical-camera integration.